

Assignment 2

This assignment covers neural networks, forward pass, backpropagation, activation functions, cross-validation techniques and vectorization.

Neural Network Library

In the first part of this assignment, you will create a neural network library. The library will be made up of documented classes and functions that allow users to easily construct a neural network with an arbitrary number of layers and nodes. Through implementing this library, you will understand more clearly the atomic components that make up a basic neural network.

The Layer Class(5points)

For the layers you will create in this assignment, it is worth it to create a parent class named **Layer** which defined the forward and backward functions that are used by all layers. In this way, we can take advantage of polymorphism to easily compute the forward and backward passes of the entire network.

Linear Layer(10points)

Create a class that implements a linear layer. The class should inherit the Layer class and implement both a forward and backward function. For a given input, the forward pass is computed as

$$f(X, W) = W^T X + b$$

Here, $X \in R^{n \times d}$, $W \in R^{h \times d}$ and $b \in R^h$, where n is the number of samples, d is the number of input features, and h is the number of output features. The backward pass should compute the gradient with respect to the weights and bias:

This is then multiplied with the gradients computed by the layer ahead of this one. Since there may be multiple layers, it should additionally compute $\frac{df}{dx}$ to complete a chain of backward passes.

$$\frac{df(x,w)}{dw} = \frac{df(x,w)}{db} = 1$$

Sigmoid Function(5points)

Create a class that implements the logistic sigmoid function. The class should inherit the Layer class and implement both forward and backward functions. It is useful to store the output of forward pass of this layer as a class member so that it may be reused when calling backward.

Rectified Linear Unit (ReLU) (5points)

Create a class that implements the rectified linear unit. The class should inherit the Layer class and implement both forward and backward functions.

Binary Cross-Entropy Loss(5points)

Create a class that implements binary cross-entropy loss. This will be used when classifying the XOR problem. The class should inherit the Layer class and implement both forward and backward functions.

Feel free to use the code defined in the class examples when integrating this with your library.

The Sequential Class(10)

In order to create a clean interface that includes multiple layers, you will need to create a class that contains a list of layers which make up the network. The Sequential class will contain a list of layers. New layers can be added to it by appending them to the current list. This class will also inherit from the Layer class so that it can call forward and backward as required.

Saving and Loading(5points)

Implement a weight saving and loading feature for a constructed network such that all model weights can be saved to and loaded from a file. This will enable trained models to be stored and shared.

Testing your library(10points)

Construct a neural network with 1 hidden layer of 2 nodes in order to solve the XOR problem. Construct the input using numpy. Train and verify that your model can solve the XOR problem. This may take many attempts to converge to a solution depending on your architecture, choice of activation function, learning rate, and other factors. Attempt to solve this problem with the architecture described above using sigmoid activations and then again using hyperbolic tangent activations. In your notebook, describe which one was easier to train for this problem. Save the weights as XOR_solved.w.

Predicting Trip Duration(10points)

In the second part of the assignment, you will use your neural network library to construct several networks for taxi trip duration ([link to dataset](#)). The original dataset does not contain the target values for the test set, so I've created a modified version based only on the test set. This is available for download on Canvas.

You can load and extract the data using the following code:

```
import numpy as np
dataset = np.load("nyc_taxi_data.npy" , allow_pickle=True).item()
X_train, y_train, X_test, y_test = dataset["X_train"], dataset["y_train"], dataset["X_test"],
dataset["y_test"]
```

Dataset Preprocessing(10points)

Some of the features in this dataset may not be well formatted for use in a neural network.

They also may not be useful for predicting the trip duration. You should experiment with different features and transformations to see which ones work best. You may also want to experiment with different normalization techniques.

In a separate document, describe the features you used and how you transformed them. Include any plots that you used to help you make your decisions.

Model Selection(10points)

In this part, you should experiment with the number of layers and nodes per layer as you wish. Use the loss of the validation set to guide your selection of hyperparameters. Experiment with at least 3 configurations of hyperparameters, plotting the training and validation loss as you train each configuration. Stop training when the loss does not improve after 3 steps (early stopping). In your notebook, include the training/validation plots with each choice of hyperparameters. Once you have trained at least 3 different models, evaluate each one on the test set. Include the test accuracy with your output.

Submission

Create a zip file that includes all relevant code (or a single notebook if applicable). The TA should be able to easily run the code to reproduce all plots and results. Include any additional instructions, if necessary.

